Análise Exploratória de Dados — VitaClin

Este notebook possui o objetivo de explorar os dados de atendimentos clínicos da VitaClin, investigando a estrutura, identificando inconsistências e visa transformar dados brutos em informação útil para o negócio, de forma organizada e visual.

Importação das Bibliotecas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')

# Deixando os gráficos mais bonitos
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.dpi'] = 110
plt.rcParams['font.family'] = 'DejaVu Sans'

print("✅ Bibliotecas importadas com sucesso!")
```

```
✅ Bibliotecas importadas com sucesso!
```

Estrutura do Dataset

- Carregando os Dados

```
import pandas as pd

df = pd.read_csv(
    'Base Vitaclin.csv',
    sep=';',
    decimal=',',
    encoding='utf-8'
)

print("✅ Arquivo carregado com sucesso!")
print(f"   O dataset possui {df.shape[0]} linhas e {df.shape[1]} colunas.")
```

```
✅ Arquivo carregado com sucesso!
   O dataset possui 5067 linhas e 9 colunas.
```

- Primeiras 5 Linhas do Dataset

```
df.head()
```

	data_atendimento	paciente_id	nomeprocedimento	grupoprocedimento	valor_
0	30/01/2024	104593.0	Rx Tornozelo Esquerdo	Raio X	
1	30/01/2024	104593.0	Rx Joelho	Raio X	
2	06/02/2024	166899.0	Consulta Clinica Geral	Clínica Geral	
3	22/02/2024	188620.0	Consulta Clinica Geral	Clínica Geral	
4	26/02/2024	45111.0	Consulta Clinica Geral	Clínica Geral	

Passos seguintes: Gerar código com df | New interactive sheet

- Dimensões do Arquivo

```
linhas, colunas = df.shape
print(f" O dataset possui {linhas} linhas e {colunas} colunas.")
print(f"\nColunas disponíveis:")
for col in df.columns:
    print(f"  • {col}")
```

```
 O dataset possui 5067 linhas e 9 colunas.

Colunas disponíveis:
  • data_atendimento
  • paciente_id
  • nomeprocedimento
  • grupoprocedimento
  • valor_pago
  • retorno
  • nomeunidade
```

- formapagamento
- idprofissional

- Tipos de Dados e Valores Nulos

```
print(" Tipos de dados por coluna:")
print(df.dtypes)
print()
print(" Valores nulos por coluna:")
nulos = df.isnull().sum()
print(nulos)
print()
print(f"Total de células nulas no dataset: {nulos.sum()}")
```

```
 Tipos de dados por coluna:
data_atendimento      object
paciente_id          float64
nomeprocedimento      object
grupoprocedimento     object
valor_pago           float64
retorno               object
nomeunidade           object
formapagamento        object
idprofissional       float64
dtype: object

 Valores nulos por coluna:
data_atendimento      0
paciente_id           7
nomeprocedimento      0
grupoprocedimento     0
valor_pago            0
retorno               0
nomeunidade           0
formapagamento        0
idprofissional       11
dtype: int64

Total de células nulas no dataset: 18
```

- Limpeza e Tratamento dos Dados

Problema encontrado	Solução aplicad
Coluna `data_atendimento` está como texto	Converter para formato de data
`nomeunidade` tem espaços extras (ex: `' VitaClin Méier'`)	Remover espaços com `.strip()`
`grupoprocedimento` normalizado para `'Clínica Geral'` em toda a base	Padronização aplicada preventivan
Linhas com `valor_pago` ou `paciente_id` nulos	Remover — não é possível analisar
Valores `valor_pago` ou `paciente_id` nulos	Remover — não é possível analisar

```
# 1. Converter data para o tipo correto
df['data_atendimento'] = pd.to_datetime(df['data_atendimento'], format='%d/%
```

```
# 2. Remover espaços extras nos nomes das unidades
df['nomeunidade'] = df['nomeunidade'].str.strip()

# 3. Garantir padronização de 'Clínica Geral' (base já normalizada)
df['grupoprocedimento'] = df['grupoprocedimento'].str.strip()  # normalizaçã

# 4. Remover linhas sem valor_pago ou paciente_id
antes = df.shape[0]
df = df.dropna(subset=['valor_pago', 'paciente_id'])
print(f"  Removidas {antes - df.shape[0]} linhas com dados essenciais faltan

# 5. Corrigir tipos dos IDs
df['paciente_id'] = df['paciente_id'].astype(int)
df['idprofissional'] = df['idprofissional'].fillna(0).astype(int)

# 6. Identificar e remover outlier de valor_pago
print("\n Outlier detectado (valor muito acima do esperado para o procedimen
print(df[df['valor_pago'] > 5000][['data_atendimento', 'nomeprocedimento', '
df = df[df['valor_pago'] <= 5000]

# 7. Criar coluna auxiliar de mês/ano para análises temporais
df['mes'] = df['data_atendimento'].dt.to_period('M')

print(f"\n Dados limpos e prontos para análise!")
print(f"   Linhas restantes: {df.shape[0]}")
print(f"   Período coberto: {df['data_atendimento'].min().date()} até {df['d
```

```
  Removidas 7 linhas com dados essenciais faltando.

 Outlier detectado (valor muito acima do esperado para o procedimento):
Empty DataFrame
Columns: [data_atendimento, nomeprocedimento, valor_pago, nomeunidade]
Index: []

 Dados limpos e prontos para análise!
   Linhas restantes: 5060
   Período coberto: 2024-01-02 até 2025-12-30
```

Fechar

```
### - Estatísticas Descritivas
```

- Estatísticas Descritivas

```
df.describe()
```

	data_atendimento	paciente_id	valor_pago	idprofissional
count	5060	5060.000000	5060.000000	5060.000000
mean	2024-12-10 09:39:59.051383552	174979.935178	31.199366	12962.939723
min	2024-01-02 00:00:00	2509.000000	0.000000	0.000000
25%	2024-04-17 00:00:00	133667.000000	9.190000	6430.000000
50%	2024-11-06 00:00:00	197498.500000	19.990000	17882.000000
75%	2025-08-01 00:00:00	228547.250000	39.990000	18122.000000
max	2025-12-30 00:00:00	294634.000000	607.000000	18664.000000
std	NaN	78252.732640	34.535086	7517.644860

Valor Total Atendido por Mês

```
# Agrupando por mês e somando os valores
valor_mes = (df.groupby('mes')['valor_pago']
               .sum()
               .reset_index())
valor_mes.columns = ['Mês', 'Valor Total (R$)']
valor_mes['Valor Total (R$)'] = valor_mes['Valor Total (R$)'].round(2)
valor_mes['Mês'] = valor_mes['Mês'].astype(str)

print(" Valor total atendido por mês:\n")
print(valor_mes.to_string(index=False))
```

```
 Valor total atendido por mês:

    Mês  Valor Total (R$)
2024-01          10276.73
2024-02           7612.13
2024-03          12329.78
2024-04           7627.93
2025-05           9444.45
2024-06           4967.07
2024-07           5969.44
2024-08           3869.89
2024-09           3390.41
2024-10           5678.58
2024-11           4202.99
2024-12           3356.42
2025-01           6168.34
2025-02           3558.26
2025-03           3124.74
2025-04           3865.94
2025-05           2831.08
2025-06           4327.15
2025-07          14160.85
2025-08           9409.25
```

```
2025-09          6971.76
2025-10          8340.67
2025-11          6947.61
2025-12          9437.32
```

```
# Gráfico de linha com área sombreada
fig, ax = plt.subplots(figsize=(14, 5))

ax.fill_between(valor_mes['Mês'], valor_mes['Valor Total (R$)'], alpha=0.12,
ax.plot(valor_mes['Mês'], valor_mes['Valor Total (R$)'],
        marker='o', color='steelblue', linewidth=2.5, markersize=6, label='V

# Rótulo em cima de cada ponto
for _, row in valor_mes.iterrows():
    ax.annotate(
        f"R${row['Valor Total (R$)']:,.0f}",
        xy=(row['Mês'], row['Valor Total (R$)']),
        xytext=(0, 10), textcoords='offset points',
        ha='center', fontsize=7.5, color='steelblue'
    )

ax.set_title(' Valor Total Atendido por Mês', fontsize=14, fontweight='bold'
ax.set_xlabel('Mês', fontsize=11)
ax.set_ylabel('Valor Total (R$)', fontsize=11)
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'R${x:,.0f}
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Insight
mes_maior = valor_mes.loc[valor_mes['Valor Total (R$)'].idxmax()]
mes_menor = valor_mes.loc[valor_mes['Valor Total (R$)'].idxmin()]
print(f"\n - Mês com maior faturamento: {mes_maior['Mês']} → R$ {mes_maior['
print(f" - Mês com menor faturamento: {mes_menor['Mês']} → R$ {mes_menor['Va
print(f" - Faturamento médio mensal: R$ {valor_mes['Valor Total (R$)'].mean(
```

```
 - Mês com maior faturamento: 2025-07 → R$ 14,160.85
 - Mês com menor faturamento: 2025-05 → R$ 2,831.08
 - Faturamento médio mensal: R$ 6,577.87
```

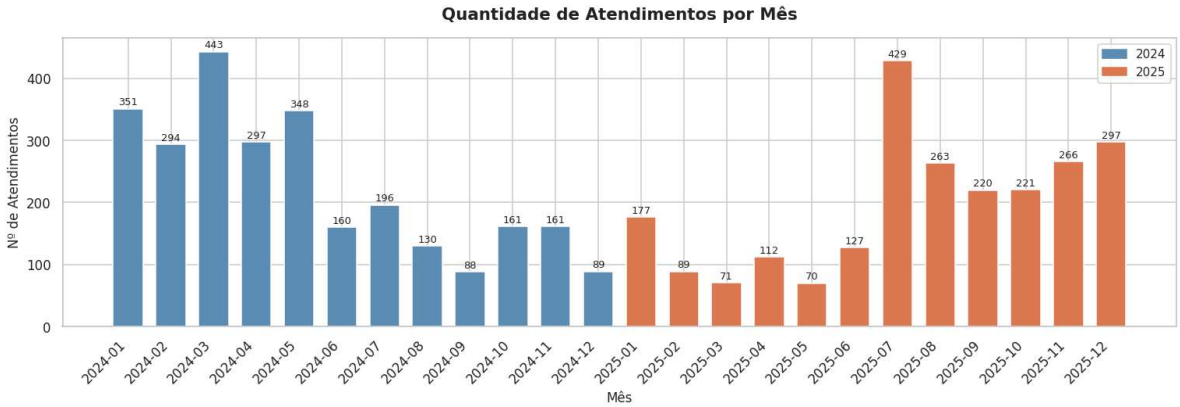
Quantidade de Atendimentos por Mês

```
# Contando atendimentos por mês
atend_mes = (df.groupby('mes')
               .size()
               .reset_index(name='Qtd Atendimentos'))
atend_mes.columns = ['Mês', 'Qtd Atendimentos']
atend_mes['Mês'] = atend_mes['Mês'].astype(str)

print(" Quantidade de atendimentos por mês:\n")
print(atend_mes.to_string(index=False))
```

```
 Quantidade de atendimentos por mês:

    Mês  Qtd Atendimentos
2024-01               351
2024-02               294
2024-03               443
2024-04               297
2024-05               348
2024-06               160
2024-07               196
2024-08               130
2024-09                88
2024-10               161
2024-11               161
2024-12                89
2025-01               177
2025-02                89
2025-03                71
2025-04               112
2025-05                70
2025-06               127
2025-07               429
2025-08               263
2025-09               220
2025-10               221
2025-11               266
2025-12               297
```

```
# Gráfico de barras — 2024 em azul, 2025 em laranja
fig, ax = plt.subplots(figsize=(14, 5))

cores = ['#e07b54' if m.startswith('2025') else '#5b8db8' for m in atend_mes
bars = ax.bar(atend_mes['Mês'], atend_mes['Qtd Atendimentos'],
              color=cores, edgecolor='white', width=0.7)

# Rótulo em cima de cada barra
for bar in bars:
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 2,
        str(int(bar.get_height())),
        ha='center', va='bottom', fontsize=8.5
```

```
    )

# Legenda
from matplotlib.patches import Patch
legenda = [Patch(color='#5b8db8', label='2024'), Patch(color='#e07b54', labe
ax.legend(handles=legenda, fontsize=10)

ax.set_title('Quantidade de Atendimentos por Mês', fontsize=14, fontweight='
ax.set_xlabel('Mês', fontsize=11)
ax.set_ylabel('Nº de Atendimentos', fontsize=11)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Insights
media = atend_mes['Qtd Atendimentos'].mean()
mes_pico = atend_mes.loc[atend_mes['Qtd Atendimentos'].idxmax()]
print(f"\n - Média mensal de atendimentos: {media:.0f}")
print(f" - Mês com mais atendimentos: {mes_pico['Mês']} ({int(mes_pico['Qtd
print(f" - Total geral de atendimentos no período: {atend_mes['Qtd Atendimen
```




```
 - Média mensal de atendimentos: 211
 - Mês com mais atendimentos: 2024-03 (443 atendimentos)
 - Total geral de atendimentos no período: 5060
```

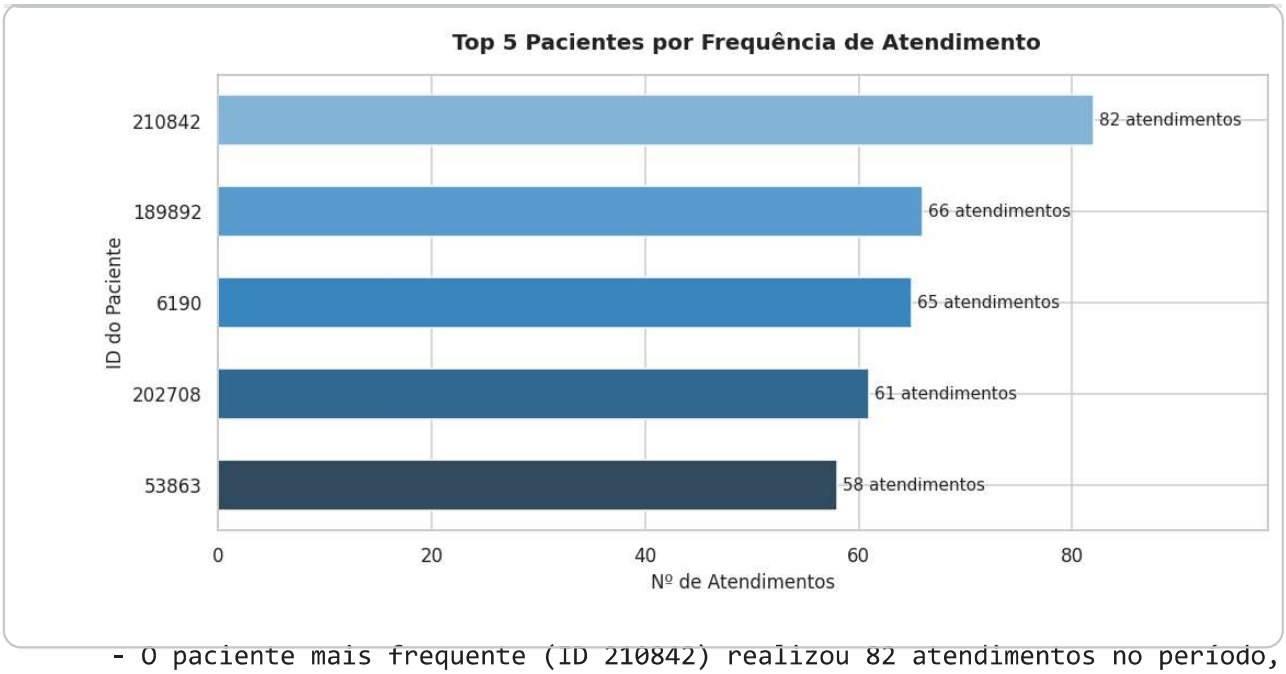
Top 5 Pacientes com Maior Frequência

```
# Agrupando por paciente com múltiplas métricas
top5 = (df.groupby('paciente_id')
          .agg(
              total_atendimentos=('paciente_id', 'count'),
              valor_total=('valor_pago', 'sum'),
              ticket_medio=('valor_pago', 'mean'),
              primeiro_atend=('data_atendimento', 'min'),
              ultimo_atend=('data_atendimento', 'max')
          )
          .sort_values('total_atendimentos', ascending=False)
          .head(5)
          .reset_index())
```

```
top5['valor_total'] = top5['valor_total'].round(2)
top5['ticket_medio'] = top5['ticket_medio'].round(2)
top5.columns = ['ID Paciente', 'Total Atendimentos', 'Valor Total (R$)',
                'Ticket Médio (R$)', 'Primeiro Atend.', 'Último Atend.']

print(" Top 5 Pacientes com Maior Frequência de Atendimento:\n")
print(top5.to_string(index=False))
```

```
 Top 5 Pacientes com Maior Frequência de Atendimento:

 ID Paciente  Total Atendimentos  Valor Total (R$)  Ticket Médio (R$) Primeir
      210842                  82           1446.47              17.64      20
      189892                  66           2357.40              35.72      20
        6190                  65           2113.57              32.52      20
      202708                  61            887.82              14.55      20
       53863                  58           1199.39              20.68      20
```

```
# Gráfico de barras horizontais
fig, ax = plt.subplots(figsize=(10, 5))

ids = top5['ID Paciente'].astype(str)
qtds = top5['Total Atendimentos']
palette = sns.color_palette('Blues_d', len(ids))

bars = ax.barh(ids, qtds, color=palette, edgecolor='white', height=0.55)

for bar, val in zip(bars, qtds):
    ax.text(
        bar.get_width() + 0.5,
        bar.get_y() + bar.get_height() / 2,
        f'{val} atendimentos',
        va='center', fontsize=10
    )

ax.set_title(' Top 5 Pacientes por Frequência de Atendimento', fontsize=13,
ax.set_xlabel('Nº de Atendimentos', fontsize=11)
ax.set_ylabel('ID do Paciente', fontsize=11)
ax.invert_yaxis()
ax.set_xlim(0, qtds.max() * 1.2)
plt.tight_layout()
plt.show()

print(f"\n - O paciente mais frequente (ID {top5.iloc[0]['ID Paciente']}) re
      f"{int(top5.iloc[0]['Total Atendimentos'])} atendimentos no período,")
print(f"  com um ticket médio de R$ {top5.iloc[0]['Ticket Médio (R$)']:,.2f}
print(f"\n - Esses 5 pacientes juntos somam {int(qtds.sum())} atendimentos –
      f"uma média de {qtds.mean():.0f} atendimentos por paciente nesse grupo
```




```
- O paciente mais frequente (ID 210842) realizou 82 atendimentos no período,
com um ticket médio de R$ 17.64 por visita.

- Esses 5 pacientes juntos somam 332 atendimentos — uma média de 66 atendime
```

Resumo dos Insights

Após a limpeza e análise dos dados, os principais achados foram:

Insight
* O dataset continha **inconsistências** em nomes de unidades (espaços extras) e grupos de procedimentos (acei
* O dataset foi tratado para garantir consistência de tipos, acentuação e espaços — base entregue já normalizac
* **Exames Laboratoriais** é a categoria mais frequente, representando mais da metade de todos os atendimentos
* O volume e o faturamento variam bastante ao longo dos meses, com picos em **março/2024** e **julho/2025**
* Os **Top 5 pacientes** têm entre 58 e 82 atendimentos cada — perfil de alta frequência que pode indicar acompa
* **Dinheiro** representa 87% da receita; **PIX** tem o maior ticket médio (R$ 35,72) apesar do menor volume

Análise exploratória — VitaClin | Jan/2024 – Dez/2025